04 CONTROLLER vs SERVICE — DIFFERENCE (NESTJS)

Both are classes Nest manages, but they answer two different questions:

```
Controller -> "WHICH request came in, and what do I send back?"
Service    -> "WHAT actually needs to happen with this data?"

  Client --HTTP--> Controller --calls--> Service --talks to--> DB/API
  Client <--JSON--- Controller <--returns-- Service <----------
```

COMPARISON TABLE

```
ASPECT                | CONTROLLER                      | SERVICE
----------------------|---------------------------------|--------------------------------
Decorator             | @Controller('path')             | @Injectable()
Job                   | Handle HTTP request/response    | Hold business logic
Knows about           | Routes, HTTP methods, status    | Domain rules, data, calculations
                      | codes, headers                  |
Contains              | Routing + delegation only       | Actual logic (validation,
                      |                                 | calculations, DB queries)
Registered in module's | controllers: [...]             | providers: [...]
Called by             | The HTTP layer (Nest/Express)   | Controllers, other services,
                      |                                 | CLI commands, cron jobs, etc.
Depends on            | One or more services (via DI)   | Other services, repositories,
                      |                                 | config, external clients
Testing style         | Test routing + mocked service   | Test real logic directly, no
                      | calls                           | HTTP involved
Should NOT contain    | Database queries, business rules | HTTP concerns (status codes,
                      |                                 | request/response shape)
Example class name    | CatsController                  | CatsService
Typical file          | cats.controller.ts              | cats.service.ts
```

SIDE-BY-SIDE EXAMPLE

cats.controller.ts

```ts
@Controller('cats')
export class CatsController {
  constructor(
    private catsService: CatsService,
  ) {}

  @Get()
  findAll() {
    return this.catsService.findAll();
  }

  @Post()
  create(@Body() dto: CreateCatDto) {
    return this.catsService.create(dto);
```

cats.service.ts

```ts
@Injectable()
export class CatsService {
  private cats: Cat[] = [];

  findAll(): Cat[] {
    return this.cats;
  }

  create(dto: CreateCatDto): Cat {
    const cat = {
      id: Date.now().toString(),
      ...dto,
    };
    this.cats.push(cat);
```

```
  }                                          return cat;
}                                          }
                                          }
```

- CatsController: knows the route is "POST /cats", knows the body is read via @Body(), knows to return whatever the service gives back as JSON. It has ZERO idea how a cat is actually created or stored.
- CatsService: knows how to generate an id, store the cat, and return it. It has ZERO idea it's being called from an HTTP POST request — it could just as easily be called from a test, a script, or a different protocol entirely.

WHY THE SPLIT MATTERS (CONCRETE SCENARIO)

Say you need to add a GraphQL API alongside your existing REST API for the same "cats" feature.

```
WITHOUT separation (logic stuck in the controller):
  You'd have to copy/rewrite the id-generation and storage logic inside
  a new GraphQL resolver — now there are two places that create cats,
  and they can drift out of sync (e.g. one validates age >= 0, the
  other forgets to).

WITH separation (logic in CatsService):
  The new GraphQL resolver just injects the SAME CatsService and calls
  .create(dto) — one source of truth for "how a cat gets created",
  reused by both REST and GraphQL.
```

QUICK MENTAL TEST

Ask yourself: "Would this code still make sense if there were no HTTP request at all — e.g. called from a script?"

- Yes → it belongs in the SERVICE.
- No (it's about status codes, headers, the URL/route) → it belongs in the CONTROLLER.